

81. Une valeur de retour : la fonction qui répond

Certaines fonctions font ; d'autres calculent, et « return » rapporte le résultat.

1. Ce qu'il faut comprendre

Remplace `void` par un type — `uint8_t` — et la fonction **rend une valeur**. `return valeur;` fait deux choses : il **termine** la fonction tout de suite, et **rapporte** la valeur à l'endroit de l'appel. `plusGrand(3, 8)` **devient** 8, comme si on l'avait écrit.

On peut donc mettre un appel **partout où un nombre est permis** : dans un calcul, dans un `nombre()`, dans un `if`, dans un autre appel.

`limiter(x, 2, 17)` rend x **ramené entre 2 et 17**. Il y a deux `return` : le premier qui s'exécute termine la fonction, les lignes d'après ne sont pas lues. C'est un style courant : **traiter les cas particuliers d'abord**, et sortir.

`dansLaZone(x)` rend 1 ou 0 : c'est une **question**, et son nom le dit. `if (dansLaZone(x))` se lit comme une phrase. Tout ce qu'on écrivait en ligne au chapitre 2 peut ainsi prendre un nom.

Dans le jeu, `x` avance **sans garde** : c'est `limiter` qui empêche de sortir. La règle est écrite **une fois**, à un seul endroit.

À toi : écris `uint8_t plusPetit(uint8_t a, uint8_t b)` et affiche `plusPetit(3, 8)`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **973 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          49 o ← la console, à l'allumage - la mise en route de la console
plusGrand()         15 o ← nombre() - ta fonction
limiter()           27 o ← main() - ta fonction
dansLaZone()        20 o ← main() - ta fonction
main()              182 o ← démarrage - ta fonction
VBlank              8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage       28 o ← image() - attend l'image suivante
AttendreVBlank      16 o ← démarrage - attend que l'écran soit libre
AttendreAcces       12 o ← EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
AdresseCase         10 o ← texte() - trouve l'adresse d'une case de l'écran
EcrireTexte         10 o ← texte() - écrit un texte, lettre par lettre
```

```

EcrireNombre      33 o ← nombre() – écrit un nombre en chiffres
CopierTuiles      103 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
LireManette       37 o ← bouton() – lit les huit boutons
Tuiles            384 o ← CopierTuiles – 24 lettres sur 43
"PLUS GRAND 3 8:" 15 o ← texte() – un texte gravé
" "               1 o ← texte() – un texte gravé
"#"               1 o ← texte() – un texte gravé
"ZONE"            4 o ← texte() – un texte gravé
" " "             4 o ← texte() – un texte gravé
Lettres gravées : ADEGLNOPRSUZ0123456789:#
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
    texte()      × 5    69 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
    nombre()     × 2    18 o sur place, dans main() + plusGrand() (15 o), EcrireNombre (33 o)
    bouton()     × 2    10 o sur place, dans main() + LireManette (37 o)
    image()      × 1     3 o sur place, dans main() + AttendreImage (28 o)
Tes fonctions appelées :
    limiter()    × 1    15 o sur place, dans main()
    plusGrand() × 1    11 o sur place, dans main()
    dansLaZone() × 1     7 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : un # bouge avec GAUCHE et DROITE, sans sortir des colonnes 2 à 17 ;
// ZONE s'affiche quand il est entre les colonnes 8 et 12.
//
// Ce qui est nouveau : une fonction qui RÉPOND (une « valeur de retour »).
// uint8_t plusGrand(...) : le mot devant le nom (uint8_t) dit ce qu'elle rend.
// return a; = « la réponse est a » ; la fonction s'arrête tout de suite.

#include <texte>    // écrit un texte à l'écran
#include <nombre>  // écrit un nombre en chiffres
#include <bouton>  // lit un bouton de la manette

```

2. UNE FONCTION — PLUSGRAND()

```

uint8_t plusGrand(uint8_t a, uint8_t b) { // Rend le plus grand de a et b.
    if (a > b) return a;                  // a est plus grand : on rend a (et on s'arrête).
    return b;                             // Sinon, on arrive ici : on rend b.
}                                          // plusGrand(3, 8) → 3 > 8 est faux → rend 8.

```

3. UNE FONCTION — LIMITER()

```

uint8_t limiter(uint8_t v, uint8_t mini, uint8_t maxi) { // Garde v entre mini et maxi.
    if (v < mini) return mini;                       // Trop petit : on rend mini.
    if (v > maxi) return maxi;                         // Trop grand : on rend maxi.
    return v;                                          // Entre les deux : v ne change pas.
}                                                    // limiter(20, 2, 17) → 17 ; limiter(1, 2, 17) → 2.

```

4. UNE FONCTION — DANSLAZONE()

```

uint8_t dansLaZone(uint8_t x) { // Rend 1 si x est entre 8 et 12, sinon 0.
    return x >= 8 && x <= 12;    // La comparaison elle-même est la réponse.
}

```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                     // Le jeu commence ici.
    texte(1, 1, "PLUS GRAND 3 8:");
    nombre(17, 1, plusGrand(3, 8));             // La réponse de la fonction s'affiche : 008.

    uint8_t x = 5;                               // La colonne du #.
    uint8_t ancien = 0;                          // Où il était dessiné avant.

    while (true) {                                // La boucle du jeu :
        image();                                  // une image.

        if (bouton(DROITE)) x++;                 // DROITE tenue : un pas à droite.
        if (bouton(GAUCHE)) x--;                 // GAUCHE tenue : un pas à gauche.
        x = limiter(x, 2, 17);                  // On range dans x la réponse de limiter :
                                                // x reste entre 2 et 17.

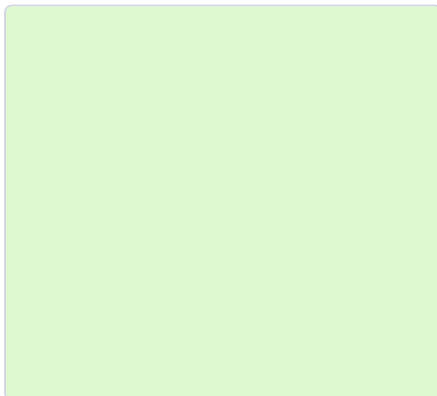
        if (x != ancien) {                      // Le # a bougé ?
            texte(ancien, 9, " ");               // efface l'ancien,
            texte(x, 9, "#");                   // dessine le nouveau,
            nombre(1, 5, x);                     // affiche la colonne.

            if (dansLaZone(x)) texte(6, 5, "ZONE"); // La réponse sert de condition : 1 ou 0.
            else texte(6, 5, "   ");             // 4 espaces pour effacer ZONE.

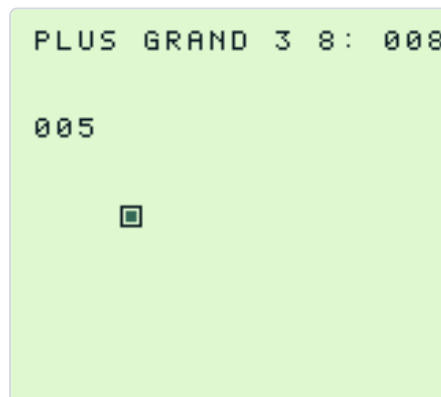
            ancien = x;                          // On retient la place.
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



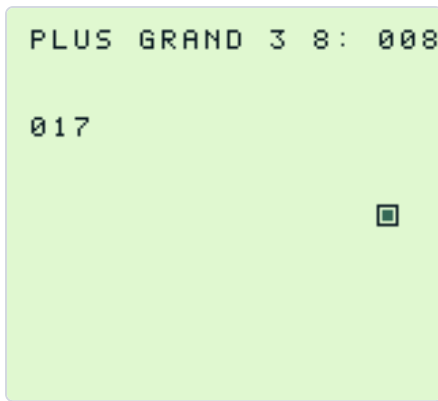
la toute première image — la console vient de démarrer — image 1



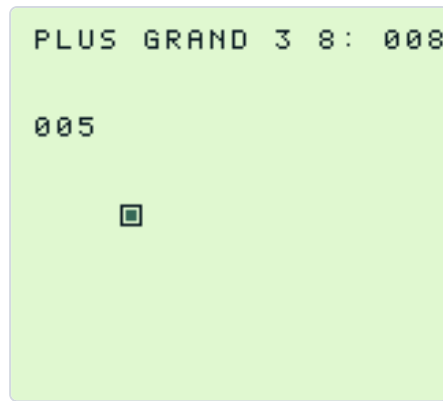
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

« PLUS GRAND 3 8: 008 », et un # que GAUCHE et DROITE déplacent sans qu'il sorte de 2..17.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ `plusGrand(3, 8)` vaut 8
- ✓ limiter l'arrêt à 17 à droite ($x = 17$)
- ✓ et à 2 à gauche, sans passer par 255 ($x = 2$)
- ✓ `dansLaZone` répond ($x = 9$)

Ensuite : **82. Découper la boucle du jeu en fonctions** — chapitre 12 : Les fonctions : nommer un geste. · `gameboy3` — écrire une cartouche Game Boy en C++